

ALPS: A Renewable, Verifiable Benchmark for Mathematical Construction Beyond Automated Reach

Anonymous submission

Abstract

Introduction

As large language models (LLMs) are applied across mathematics and the sciences, AI-for-science has grown increasingly interested in evaluating LLM reasoning and creativity. Recently, automated mathematical reasoning has become one of the most visible frontiers within AI-for-science, with systems now reaching gold-medal performance at the International Mathematical Olympiad (Google DeepMind 2025). Two major recent developments shift this attention specifically towards generating proofs. In 2026, a general-purpose reasoning model resolved the roughly eighty-year-old Erdős unit-distance problem, a long-open question in discrete geometry (OpenAI 2026). The SAIR Foundation’s Mathematics Distillation Challenge (Tao 2026), organized by Terence Tao and Damek Davis, asks competitors to design LLM systems that can construct proofs over the algebraic questions of the Equational Theories Project (Bolan et al. 2025). The common thread is a standard that the rest of AI-for-science rarely meets: a creative answer only counts once it has been proven correct.

Verifiability of this kind is just one component necessary for a rigorous benchmark. As LLMs begin to tackle scientific work - formalizing and proving theorems (Yang et al. 2023; Xin et al. 2024), proposing conjectures (Romera-Paredes et al. 2024; Gottweis et al. 2026; Lu et al. 2024), and designing molecules and materials (Jumper et al. 2021; Merchant et al. 2023) - the essential question becomes whether what they produce is genuinely new. However, current benchmarks fail to sufficiently measure this capability. Judging LLM outputs using human experts or through physical experiment is slow, costly, and subjective. Scoring against reference answers provides an objective measurement, but such answers often contaminate LLM training data (Xu et al. 2024; Chen et al. 2025). This makes it difficult to discern whether a high score reflects recall rather than discovery. What is missing is

a benchmark that scores novel work objectively, on problems no LLM could have seen before.

Problems of this kind exist within equational theory (Birkhoff 1935). This area of mathematics studies magmas: sets with a single binary operation $\diamond$ that combines two elements into a third (Burris, Sankappanavar et al. 1981). Its laws are universally quantified equations that an operation may or may not obey, such as commutativity $x \diamond y = y \diamond x$ (Burris, Sankappanavar et al. 1981; Baader and Nipkow 1998). A magma is a model of a law when the equation holds under every assignment of its elements (Burris, Sankappanavar et al. 1981). The one-element magma satisfies every law and is always a model. If there exists only one element, all variables must always be equivalent to each other. Therefore, this model is never informative, and is therefore named the trivial magma (Burris, Sankappanavar et al. 1981). What carries content is whether a law has a nontrivial model, one with at least two elements. On a finite carrier, such a model can simply be found by search, since only finitely many operations exist to test. However, some laws have no nontrivial finite model at all. Their only nontrivial models are infinite, and no search through finite candidates can reach an infinite structure (Kisielewicz 1997; Bolan et al. 2025). Here, the questions stop being a matter of computation and become a matter of reasoning about the magma’s structure as a whole.

We build the benchmark from the laws that require this reasoning: those with no nontrivial finite model, a property that we verify in advance. With finite models excluded, each law falls into one of two cases - either it forces every model to be trivial, entailing $x = y$, or it has a nontrivial model, which must then be infinite. Laws of the second kind, an infinite model but no finite one, are the Austin laws (Kisielewicz 1997).

Austin laws sit where AI-for-scientists and mathematical provers are converging. The recent Erdős unit-distance result was settled by a reasoning model through the construction of an infinite object, the kind of scientific leap that marks the frontier of what today’s language models can do (OpenAI 2026). Computational mathematical provers approach this same frontier from the other side, with recent progress demonstrating new methods of identifying proofs for infinite models (Janota, Rawson, and Schulz 2026). Austin laws are this challenge in a controlled, renewable form: each is an infinite-construction problem with a definite answer, and

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

fresh ones can be generated without limit. It is that combination, a frontier problem that is nonetheless checkable and endless in supply, that makes them something a benchmark can be built upon.

We call this benchmark Austin-Law Proof-Synthesis (ALPS). Each problem is one such law, and solving one means providing a proof showing which of the two cases holds true. A solver must either establish that the law is Austin, by constructing an infinite model and showing it satisfies the law, or prove that the law entails $x = y$, meaning that all models for this law must be trivial.

Three properties set ALPS apart, shown as the three overlapping ovals in Figure 1, where each benchmark family lies within the region matching the properties it has. ALPS is verifiable: each answer is settled by a certificate a machine can check, so grading is human-free and provably correct. It is renewable: fresh instances can be produced without limit, so evaluation always has problems no model has seen. And it demands creative construction: a nontrivial model exists only among infinite structures and cannot be reached with a general procedure, so solutions must be invented for each law. Every prior benchmark family has at most two such properties. Synthetic puzzles such as random SAT (Selman, Mitchell, and Levesque 1996) and the Equational Theories Project (Bolan et al. 2025) are verifiable and renewable, but their problems are solved by a general procedure given enough compute, not by construction. Static corpora such as MATH (Hendrycks et al. 2021), coding benchmarks (Chen et al. 2021), and miniF2F (Zheng, Han, and Polu 2021), ask for constructed, checkable solutions. However, these come from fixed sets that eventually contaminate without a clear way of generating high-quality new problems. Open scientific discovery (Lu et al. 2024; Gottweis et al. 2026) demands construction and never exhausts its supply, but its answers are judged subjectively. ALPS alone lies where all three meet.

What ALPS requires of the solver is a capability that reaches well beyond equational theory. Resolving a law requires proposing a completely novel structure, then arguing from the law alone that it satisfies a specification over infinitely many cases it can never check directly. This mirrors mathematical discovery and the building of scientific theories: the answer can never be searched for or recalled, only hypothesized and then justified. What sets ALPS apart is that here the justification is a formal proof, so the creative leap is isolated and made objectively measurable. These problems are exceedingly difficult, but that is the intent: progress on them is direct evidence of the ability to construct and verify genuinely new structure - the capability that separates discovery from recall, and the one current benchmarks cannot measure.

To evaluate the true difficulty of this benchmark, we construct a portfolio of the strongest automated provers (Kovács and Voronkov 2013; Schulz, Cruanes, and Vukmirović 2019; Smallbone 2021), finding that it resolves only a small part of ALPS. A large and growing set of problems resists it entirely, and on these it constructs no model at all, at any compute budget, even though every instance is certified to have a determinate answer. This difficulty is bound by method, not computational budget. In full, our contributions are:

- a two-sided construction task on Austin laws, with every instance certified to have a determinate answer;
- a corpus of such laws and a public generator that mints fresh instances without limit;
- an automated judge that certifies submissions through independent channels - a Lean proof (Moura and Ullrich 2021) for triviality, and an automated-prover certificate (Kovács and Voronkov 2013) for constructed models - with no human in the loop; and
- a strong automated baseline that establishes the method-bound hard tier and, along the way, settles an order-5 case left open by the Equational Theories Project (ETP) (Bolan et al. 2025).

Related Works

AI-for-Science Benchmarks

Existing benchmark families each hold two of the three properties ALPS targets and fall short on the third. Static problem sets are fixed collections spanning grade-school and competition mathematics (Cobbe et al. 2021; Hendrycks et al. 2021), graduate science (Rein et al. 2023), coding (Chen et al. 2021), and formal proof construction (Zheng, Han, and Polu 2021; Azerbayev et al. 2023; Tsoukalas et al. 2024). These are scored objectively and require reasoning through each problem rather than recalling a memorized algorithm. However, because their problems are fixed, these benchmarks inevitably suffer from contamination, as their evaluation set will eventually enter the training data of the language models they are meant to test, rendering further evaluation on this benchmark unreliable (Xu et al. 2024; Chen et al. 2025). Some benchmarks attempt to delay contamination by holding problems secret (Glazer et al. 2024) or releasing new problems over time (White et al. 2025; Jain et al. 2025), but will always be limited by the rate at which humans can author new problems.

Synthetic puzzles avoid this limit by generating problems from a fixed pattern or rule so that fresh instances can be produced without end, such as random SAT formulas (Selman, Mitchell, and Levesque 1996), graph-generated reasoning tasks (Zhu et al. 2024), Sudoku puzzles (Seely et al. 2025), or the equational laws of the Equational Theories Project (Bolan et al. 2025). However, the same regularity that makes these problems generable also makes them solvable by a known procedure. Therefore, these benchmarks often test the LLM’s ability to execute a known method rather than to truly construct a creative solution.

Open scientific discovery benchmarks evaluate an LLM’s ability to construct novel, useful scientific advancements such as hypotheses, conjectures, designs, and mathematical structures (Lu et al. 2024; Gottweis et al. 2026; Alkan et al. 2025; Romera-Paredes et al. 2024; Merchant et al. 2023; Jumper et al. 2021). These pose a challenging task, requiring the LLM to invent a new solution rather than retrieve or execute a known procedure. However, their results are validated by human expertise or physical experiment, making evaluation subjective and slow. ALPS uniquely combines the strengths of each benchmark family: objective scoring, endless renewal, and a demand for genuine construction.

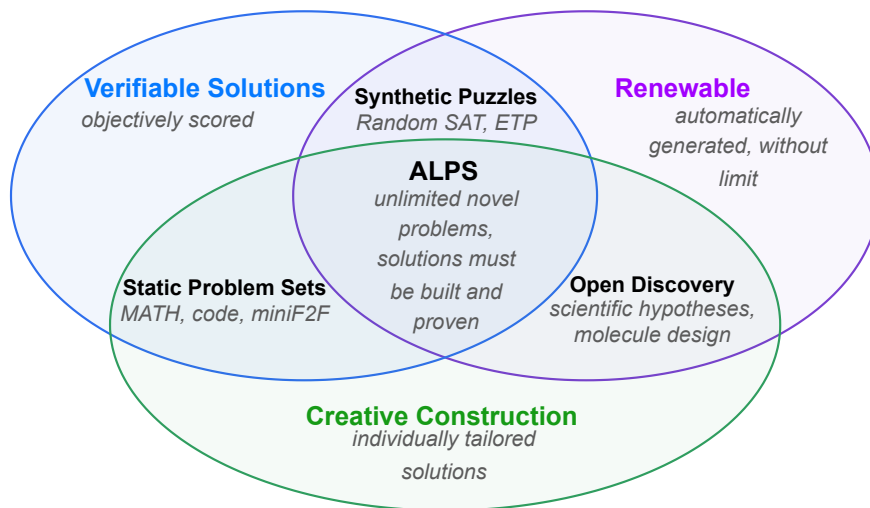


Figure 1: The three properties ALPS combines - objective scoring, renewability, and creative construction - shown as three overlapping regions. Each family achieves at most two: static problem sets such as MATH, code, and miniF2F (Hendrycks et al. 2021; Chen et al. 2021; Zheng, Han, and Polu 2021) are scored and demand construction but contaminate; synthetic puzzles such as random SAT and the Equational Theories Project (Selman, Mitchell, and Levesque 1996; Bolan et al. 2025) are scored and renewable but follow a known procedure; open scientific discovery such as hypothesis generation and molecule design (Gottweis et al. 2026; Jumper et al. 2021) constructs and renews but is judged subjectively. Only ALPS lies in the intersection of all three.

Theorem Proving and Verification

ALPS scores answers using machine verification, without a human in the loop, made possible by theorem provers. Lean (Moura and Ullrich 2021) is a functional programming language equipped with a proof assistant whose kernel can automatically check the validity of proofs, such that a certificate is guaranteed to be correct. Each proposition, including the statement that a magma satisfies a law, is encoded as a type within the language, and a proof of it is a term inhabiting that type, exactly as a value inhabits a type in an ordinary programming language. Checking a proof is resolved using type checking: the kernel confirms that submitted terms have the claimed type, similar to a compiler checking that a function’s body matches its signature. A statement quantified over infinitely many inputs, such as a law holding under every assignment, becomes a function type. Within Lean, its proof is a function returning a proof for an arbitrary argument, and the kernel type-checks that function once, symbolically. A certificate that type-checks successfully is therefore correct by construction and automatically validated.

Unlike checking them, finding proofs is difficult. Many works investigate automating this search. Classical automated theorem provers such as Vampire and E (Kovács and Voronkov 2013; Schulz, Cruanes, and Vukmirović 2019) search for first-order proofs by superposition calculus (Bachmair and Ganzinger 1994), a strategy that repeatedly derives logical consequences of the input until it reaches a contradiction or no new consequences remain. More recently, language models have been trained to construct proofs in systems like

Lean (Yang et al. 2023; Xin et al. 2024; Hubert et al. 2026), measured on formal proof benchmarks (Zheng, Han, and Polu 2021).

Equational Reasoning

Equational theory studies the foundation of algebra - the identities that operations satisfy and the entailments among them. The Equational Theories Project (Bolan et al. 2025) studies this at scale, enumerating the implications among thousands of small laws and settling them automatically using a pipeline of automated provers and finite-model finders. This work singles out the laws whose only nontrivial models are infinite, the Austin laws (Kisielewicz 1997), as the hardest to resolve, since neither a searchable finite model nor a short derivation settles them.

Several automated methods are tailored to equational laws. Paradox and Mace4 (Claessen and Sörensson 2003; McCune 2003) are designed to find finite models when they exist, while Infinox (Claessen and Lillieström 2011) instead proves that no finite model exists. Twee (Smallbone 2021) implements unfailing Knuth-Bendix completion (Knuth and Bendix 1970; Bachmair, Dershowitz, and Plaisted 1989), orienting equations into rewrite systems to solve equational problems. When a saturation from Twee or a superposition prover halts without reaching a contradiction, the clauses it has accumulated already describe a model. Janota, Rawson, and Schulz (2026) recently studied this for equational theories, extracting an explicit model from the saturation in a case study.

The Austin-Law Proof-Synthesis Benchmark

Problem Formulation

A magma is a set M , called the carrier, together with a single binary operation $\diamond: M \times M \rightarrow M$. An equational law is a universally quantified identity between two terms built from variables and $\diamond$, such as the commutative law $x \diamond y = y \diamond x$ or the associative law $(x \diamond y) \diamond z = x \diamond (y \diamond z)$. The order of a law is the number of times the operation appears in it. A magma satisfies a law when the identity holds under all possible assignments of carrier elements to the variables (addition and multiplication, for example, satisfy both the commutative and associative laws). The object of interest for any law is the set of magmas in which it holds. Our benchmark is built from laws of the form $L: x = T[x, y, z, \dots]$, a single variable equated with a term T of order five or more over the operation. A magma with a carrier of size 1 satisfies every law automatically: with only one element to assign, the identity $x = y$ can never fail, and such a magma is called trivial. The question the benchmark poses for each L is whether L entails $x = y$, forcing every magma that satisfies it to be trivial, or whether L admits a nontrivial model, one whose carrier holds at least two distinct elements.

A nontrivial model, when one exists, may be finite or infinite. A finite model can always be found by searching through all magmas of increasing size until it reaches a satisfying assignment, requiring none of the creative, law-specific construction the benchmark aims to measure. We therefore admit a law into the benchmark only if it carries a machine-checked certificate that it has no nontrivial finite model; we call such a law admissible. For these laws, any nontrivial model that exists must be infinite, forcing a clean dichotomy: the law either entails $x = y$ and is trivial, or it is satisfied by an infinite, nontrivial magma. Laws that possess infinite models but no nontrivial finite one are known as Austin Laws (?). Every law either entails $x = y$ or it does not, so each instance has a determinate answer. Admissibility guarantees that the two possibilities are exactly the trivial and the Austin case. No instance is therefore unanswerable, and its difficulty lies in finding the answer, not in whether one exists.

The two sides of the task are not symmetric, and this asymmetry is essential to what the benchmark measures. Whether L entails $x = y$ is a first-order consequence, and a complete proof search will confirm it whenever it holds. The trivial case is, in principle, settled by deduction. The Austin case admits no such guarantee. A proof search that fails to derive $x = y$, however long it runs, does not show that L is Austin, since the absence of a derivation is not a model, and since equational entailment is undecidable in general (?), no uniform procedure can be expected to solve all Austin laws. Establishing that a law is Austin ultimately means constructing an infinite magma and proving that it satisfies the law. Such a magma is not simply selected; it must be invented: an infinite carrier and operation must be posited and then shown to obey the identity. This is the capability the benchmark is designed to isolate: the synthesis of a new mathematical structure, tailored to the individual law, that deductive search does not supply.

Answer Verification

Given an admissible law L , a solver must produce an answer that resolves the dichotomy and that a machine can check. This submission takes one of two forms: a model of L , a carrier and an operation on it, together with a proof that the model satisfies the law and is nontrivial, or a proof that L entails $x = y$. Both can be checked automatically by the proof assistant Lean (?), and a submission is accepted only when that check succeeds. A solved instance is therefore a verified fact rather than a predicted label, and scoring the benchmark requires no human in the loop.

To make the solution for each law L checkable, we generate two propositions a solver may prove. Constructing a model means proving

$$\text{AUSTINGOAL} \equiv \exists M \exists \diamond: M \times M \rightarrow M, \\ (\exists a, b : M, a \neq b) \wedge \text{Law}(\diamond).$$

an existential over a carrier M , an operation on it, and a witness that two of its elements differ, where $\text{Law}(\diamond)$ represents the assertion that an operation $\diamond$ satisfies L . Proving the law trivial means proving

$$\text{TRIVIALGOAL} \equiv \forall M \forall \diamond: M \times M \rightarrow M, \\ \text{Law}(\diamond) \rightarrow \forall a, b : M, a = b.$$

The nontriviality clause $\exists a, b. a \neq b$ is what separates a genuine model from the one-element magma. Together with the admissibility certificate, it forces that model to be infinite. Notably, a nontrivial model need not be infinite, whereas Austin laws are defined by their infinite models. However, admissibility has already excluded every nontrivial finite model, so any magma that satisfies $\text{Law}(\diamond)$ with two distinct elements is necessarily infinite, and a proof of AUSTINGOAL therefore establishes that L is an Austin law. Requiring only nontriviality keeps the goal tractable, since the solver never has to prove infinitude inside Lean.

Corpus Construction

Our construction produces admissible laws in bulk while making the corpus hard to resolve, internally distinct, and novel. We build the corpus by extension, growing it outward from laws already known to be Austin: each seed is extended by a single operation to yield laws of the next order, which are then screened for quality. This makes hard laws practical to produce at scale and the corpus renewable, since the same construction applies recursively at every order, keeping it beyond the frontier of previously enumerated laws, where no model could have encountered it during training. Random sampling, the obvious alternative, is ineffective: admissible laws are too rare to appear in quantity, and the few that do are often redundant or already enumerated.

Concretely, we generate new candidates by extending a seed Austin law $x = T$ by selecting some variable v within T and replacing it with $v \diamond w$ or $w \diamond v$, where w is some other variable. Because the extended law inherits most of the seed's term structure, it often inherits the mechanism that denies the seed any nontrivial finite model, which is why extension yields admissible laws far more often than sampling at random.

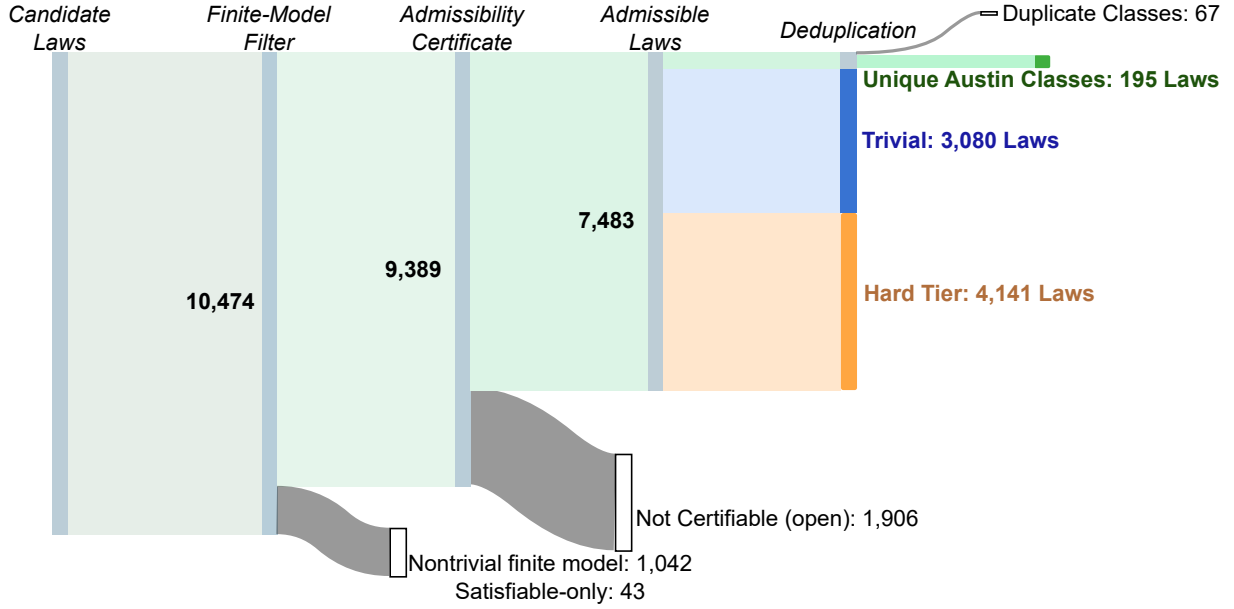


Figure 2: Methodology Figure - placeholder caption

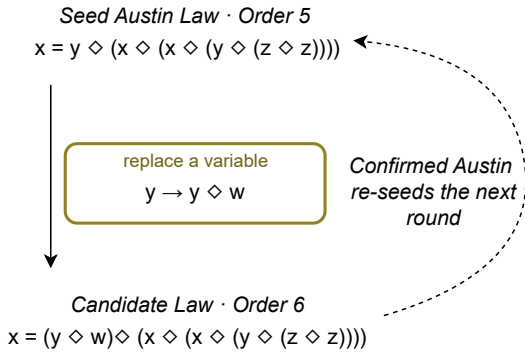


Figure 3: Methodology Figure 2 - placeholder caption

Next, each candidate passes through a sequence of screens ordered from cheap to expensive. First, a fast proof search prunes the candidates it can prove trivial. The rest are passed to the admissibility prover, a first-order query whose success is a proof that the law has no nontrivial finite model: when some subterm of T contains every occurrence of x , that subterm acts as a left inverse and makes the map $x \mapsto T$ injective. On a finite carrier, an injective map is also surjective, and asserting that surjectivity lets the prover derive $x = y$, so every finite model is trivial. This is essential, as it means admission requires an actual proof that no nontrivial finite model exists. No such proof can exist for a law that has one, so it is impossible for a finite-model law to enter the corpus. The cost is that the query is sound but incomplete.

When the occurrences of x are split across the operation at the root, it certifies nothing. These candidates are conservatively discarded even though some are genuinely admissible. What remains are the admissible laws, certified to have no nontrivial finite model, and therefore either trivial or Austin.

Because every law originates from a small set of seeds, the corpus risks redundancy, as extensions may be logically equivalent to its seed or to a sibling. We therefore measure distinctness across the corpus. Two laws are equivalent when each entails the other, which in general, demands a prover for every pair. For the laws already shown to be Austin, the models found in that process can be compared with no prover call at all. A magma in which one law holds and the other fails is already a certificate of distinctness. Each separation is a certificate that proves distinctness: a magma in which one law holds and the other fails proves the two laws distinct. Of the 34,191 pairs across 262 laws proven to be Austin, 33,936 are separated by such a model, with only 255 proved equivalent by the prover, with none left undecided. Therefore, the laws collapse to 195 classes, a reduction of roughly a quarter. We therefore report the corpus size in classes rather than laws to ensure the count reflects distinct problems.

From our observations, extension keeps producing new distinct classes. Across the orders we have only a minority of extensions are redundant, so each round yields many laws that no earlier one is equivalent to. We quantify this as the number of distinct classes, measured after admissibility and deduplication. Because this yield stays high and extension has no ceiling on order, the corpus grows without repeating itself. This makes the benchmark renewable and, in turn, contamination-free. Evaluation can always draw on instances minted after the model under test was trained, something

Candidate outcome	Count
<i>Admitted (certified no nontrivial finite model)</i>	
Austin, model exhibited	247
Admissible, unresolved (hard tier)	3,798
Trivial ($L \models x=y$)	2,895
<i>Discarded / uncertified</i>	
Has a nontrivial finite model	990
Not certified admissible	1,795
Total candidates	9,725

Table 1: Composition of the generated pool. A candidate is *admitted* only on a proof that it has no nontrivial finite model; admitted laws are each trivial or Austin, and the *hard tier* is the admissible subset the baseline leaves unresolved. Counts are by distinct law string; modulo equivalence the corpus is smaller (Table 2).

Distinctness (Austin-proven sample)	
Laws compared	262
Pairs separated prover-free	1,123/1,128
Equivalences confirmed (merges)	67
Distinct classes (upper bound)	≤ 195
Collapse	$\approx 26\%$

Table 2: Distinctness on the Austin-proven laws. Most pairs are *separated* by an explicit model in which one law holds and the other fails—a certificate that they are distinct—so the class count is anchored from below; the few unresolved pairs are counted as distinct, making 195 an upper bound. The corpus is reported in classes, not laws. Pair figures shown for the 48-law pilot; the 262-law run gives the class count.

a fixed benchmark cannot offer, since once released, it is eventually absorbed into training data.

Experiments

Baseline Portfolio

We establish the difficulty of the benchmark against a strong portfolio of automated off-the-shelf provers. Because the task is two-sided, the portfolio must be as well. Proving a law trivial requires a proof search, while finding a nontrivial model is completed through saturation. The portfolio spans eight configurations: Vampire 5.0.1 in five modes (proof search and saturation, under both Knuth-Bendix ordering and lexicographic path ordering (?), the E prover in a proof and a saturation mode (?), and Twee 2.6.1, an implementation of unfailing completion built for equational reasoning (?). Each configuration is run over a ladder of per-prover time budgets, 30, 60, 120, 300, and 600 seconds, and a law counts as resolved as soon as any configuration returns a verdict. Provers, versions, and exact flags are pinned in a released container image. A claimed model or triviality proof is one the tool itself reports rather than one we infer. We deliberately omit finite-model finders such as Mace4 and Paradox, as admissibility certifies that no nontrivial finite model exists, so these

Pass / budget	laws resolved	models built
Filtering (30s, 1 config, full pool)	3,342/10,474	262
Portfolio 30s (8 configs, hard sample)	3/120	0
60s	0/117	0
120s	0/117	0
300s	0/117	0
600s	0/117	0

Table 3: The filtering pass sorts the full pool at 30 s per stage on a single prover, building every one of the 262 models (a median of 0.1 s each) and leaving 4,141 laws unresolved. Re-attacking a 120-law sample of that residual with eight configurations at up to 600 s each—about $170\times$ the per-law time and a comparable 231 versus 262 core-hours overall—yields three trivial resolutions and not one model. Models are found cheaply or never.

tools cannot succeed. The portfolio’s saturation and completion model are instead the automated methods that are able to attempt infinite-model construction.

The hard tier is the set of admissible laws the filtering pass does not resolve. Its difficulty is bound by method rather than computation budget, demonstrated in Table 3. The filtering pass runs a single prover for thirty seconds, building 262 models each in a median of 0.1 seconds. We then reanalyze a sample of the residual with the full eight-configuration portfolio across the budget ladder. If compute were the limit, we would expect resolution to climb as the budget grows. Instead, on 120 laws, the portfolio solves three at the shortest budget, and none thereafter, even at twenty times the time per configuration. under every configuration as a “hard” tier of laws.

The three laws it does settle were all proved trivial and were not yet shown to entail $x = y$ by the short filter. These are contamination that the stronger portfolio removes, not models it builds. The construction side of the hard tier is untouched by the strongest automated methods we can assemble.

Our solver portfolio is built to contain provers that reason in different ways. Vampire and E are superposition provers: they search by deriving logical consequences of the law until either a contradiction proves it trivial or the consequences close into a model. Twee instead runs unfailing completion, the equational method behind Knuth-Bendix rewriting (?). Twee turns the law’s equations into rewrite rules and completes them into a convergent system. These are two major, distinct paradigms for equational reasoning, each exploring the space differently enough that a problem one diverges on, the other may finish. Twee is also purpose-built for the unit-equational setting our laws fall in. In our experiments, Twee resolves an order-five law in seconds whose saturation runs Vampire to hundreds of clauses. However, across the hard tier, Twee resolves nothing the rest of the portfolio does not, and produces no model of its own. Consistent failures across both automated proof paradigms suggests that the hard frontier’s difficulty is shared across automated provers as a class, not particular to any one.

LLM Baseline

Todo: Discuss LLM evaluation protocol, show these problems are hard for LLMs too, eval is fair by construction: all LLMs get the same "jumping-off" point to reason from.

Case Study

Todo: Our pipeline solves a few open problems mentioned in the ETP paper. We can discuss these two problems, showing that they are confirmed Austin, and how they are solved to give a better understanding of the types of problems we provide.

Discussion

References

- Alkan, A. K.; Sourav, S.; Jablonska, M.; Astarita, S.; Chakrabarty, R.; Garuda, N.; Khetarpal, P.; Pióro, M.; Tanoglidis, D.; Iyer, K. G.; et al. 2025. A survey on hypothesis generation for scientific discovery in the era of large language models. *arXiv preprint arXiv:2504.05496*.
- Azerbayev, Z.; Piotrowski, B.; Schoelkopf, H.; Ayers, E. W.; Radev, D.; and Avigad, J. 2023. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*.
- Baader, F.; and Nipkow, T. 1998. *Term rewriting and all that*. Cambridge university press.
- Bachmair, L.; Dershowitz, N.; and Plaisted, D. A. 1989. Completion without failure. In *Rewriting Techniques*, 1–30. Elsevier.
- Bachmair, L.; and Ganzinger, H. 1994. Rewrite-based equational theorem proving with selection and simplification. *Journal of Logic and Computation*, 4(3): 217–247.
- Birkhoff, G. 1935. On the structure of abstract algebras. In *Mathematical proceedings of the Cambridge philosophical society*, volume 31, 433–454. Cambridge University Press.
- Bolan, M.; Breitner, J.; Brox, J.; Carlini, N.; Carneiro, M.; van Doorn, F.; Dvorak, M.; Goens, A.; Hill, A.; Husum, H.; et al. 2025. The Equational Theories Project: Advancing collaborative mathematical research at scale. *arXiv preprint arXiv:2512.07087*.
- Burris, S.; Sankappanavar, H. P.; et al. 1981. *A course in universal algebra*, volume 78. Springer.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. D. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Chen, S.; Chen, Y.; Li, Z.; Jiang, Y.; Wan, Z.; He, Y.; Ran, D.; Gu, T.; Li, H.; Xie, T.; et al. 2025. Recent advances in large language model benchmarks against data contamination: From static to dynamic evaluation. *arXiv preprint arXiv:2502.17521*.
- Claessen, K.; and Lillieström, A. 2011. Automated inference of finite unsatisfiability. *Journal of Automated Reasoning*, 47(2): 111–132.
- Claessen, K.; and Sörensson, N. 2003. New techniques that improve MACE-style finite model finding. In *Proceedings of the CADE-19 Workshop: Model Computation-Principles, Algorithms, Applications*, 11–27.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Glazer, E.; Erdil, E.; Besiroglu, T.; Chicharro, D.; Chen, E.; Gunning, A.; Olsson, C. F.; Denain, J.-S.; Ho, A.; Santos, E. d. O.; et al. 2024. Frontiermath: A benchmark for evaluating advanced mathematical reasoning in ai. *arXiv preprint arXiv:2411.04872*.
- Google DeepMind. 2025. Advanced Version of Gemini with Deep Think Officially Achieves Gold-Medal Standard at the International Mathematical Olympiad. <https://deepmind.google/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-mathematical-olympiad/>.
- Gottweis, J.; Weng, W.-H.; Daryin, A.; Tu, T.; Sirkovic, P.; Myaskovsky, A.; Glowaty, G.; Weissenberger, F.; Orlandi, A.; Popovici, D.; et al. 2026. Accelerating scientific discovery with Co-Scientist. *Nature*, 1–3.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.
- Hubert, T.; Mehta, R.; Sartran, L.; Horváth, M. Z.; Žužić, G.; Wieser, E.; Huang, A.; Schrittwieser, J.; Schroecker, Y.; Ma-soom, H.; et al. 2026. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106): 607–613.
- Jain, N.; Gu, A.; Li, W.-D.; Yan, F.; Zhang, T.; Wang, S.; Solar-Lezama, A.; Sen, K.; and Stoica, I. 2025. Live-codebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, 58791–58831.
- Janota, M.; Rawson, M.; and Schulz, S. 2026. Case Study: Saturations as Explicit Models in Equational Theories. *arXiv preprint arXiv:2602.16324*.
- Jumper, J.; Evans, R.; Pritzel, A.; Green, T.; Figurnov, M.; Ronneberger, O.; Tunyasuvunakool, K.; Bates, R.; Žídek, A.; Potapenko, A.; et al. 2021. Highly accurate protein structure prediction with AlphaFold. *nature*, 596(7873): 583–589.
- Kisielewicz, A. 1997. Austin identities. *algebra universalis*, 38(3): 324–328.
- Knuth, D. E.; and Bendix, P. B. 1970. Simple word problems in universal algebras. In *Computational problems in abstract algebra*, 263–297. Elsevier.
- Kovács, L.; and Voronkov, A. 2013. First-order theorem proving and Vampire. In *International Conference on Computer Aided Verification*, 1–35. Springer.
- Lu, C.; Lu, C.; Lange, R. T.; Foerster, J.; Clune, J.; and Ha, D. 2024. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.

- McCune, W. 2003. Mace4 reference manual and guide. *arXiv preprint cs/0310055*.
- Merchant, A.; Batzner, S.; Schoenholz, S. S.; Aykol, M.; Cheon, G.; and Cubuk, E. D. 2023. Scaling deep learning for materials discovery. *Nature*, 624(7990): 80–85.
- Moura, L. d.; and Ullrich, S. 2021. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, 625–635. Springer.
- OpenAI. 2026. An OpenAI Model Has Disproved a Central Conjecture in Discrete Geometry. <https://openai.com/index/model-disproves-discrete-geometry-conjecture/>.
- Rein, D.; Hou, B. L.; Stickland, A. C.; Petty, J.; Pang, R. Y.; Dirani, J.; Michael, J.; and Bowman, S. R. 2023. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*.
- Romera-Paredes, B.; Barekatin, M.; Novikov, A.; Balog, M.; Kumar, M. P.; Dupont, E.; Ruiz, F. J.; Ellenberg, J. S.; Wang, P.; Fawzi, O.; et al. 2024. Mathematical discoveries from program search with large language models. *Nature*, 625(7995): 468–475.
- Schulz, S.; Cruanes, S.; and Vukmirović, P. 2019. Faster, higher, stronger: E 2.3. In *International Conference on Automated Deduction*, 495–507. Springer.
- Seely, J.; Imajuku, Y.; Zhao, T.; Cetin, E.; and Jones, L. 2025. Sudoku-Bench: Evaluating creative reasoning with Sudoku variants. *arXiv preprint arXiv:2505.16135*.
- Selman, B.; Mitchell, D. G.; and Levesque, H. J. 1996. Generating hard satisfiability problems. *Artificial intelligence*, 81(1-2): 17–29.
- Smallbone, N. 2021. Twee: an equational theorem prover. In *International Conference on Automated Deduction*, 602–613. Springer.
- Tao, T. 2026. Mathematics Distillation Challenge – Equational Theories. <https://terrytao.wordpress.com/2026/03/13/mathematics-distillation-challenge-equational-theories/>.
- Tsoukalas, G.; Lee, J.; Jennings, J.; Xin, J.; Ding, M.; Jennings, M.; Thakur, A.; and Chaudhuri, S. 2024. Putnam-bench: Evaluating neural theorem-provers on the putnam mathematical competition. *Advances in Neural Information Processing Systems*, 37: 11545–11569.
- White, C.; Dooley, S.; Roberts, M.; Pal, A.; Feuer, B.; Jain, S.; Schwartz-Ziv, R.; Jain, N.; Saifullah, K.; Dey, S.; et al. 2025. Livebench: A challenging, contamination-limited llm benchmark. In *International Conference on Learning Representations*, volume 2025, 91595–91631.
- Xin, H.; Guo, D.; Shao, Z.; Ren, Z.; Zhu, Q.; Liu, B.; Ruan, C.; Li, W.; and Liang, X. 2024. Deepseek-prover: Advancing theorem proving in llms through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*.
- Xu, C.; Guan, S.; Greene, D.; Kechadi, M.; et al. 2024. Benchmark data contamination of large language models: A survey. *arXiv preprint arXiv:2406.04244*.
- Yang, K.; Swope, A.; Gu, A.; Chalamala, R.; Song, P.; Yu, S.; Godil, S.; Prenger, R. J.; and Anandkumar, A. 2023. Leandojo: Theorem proving with retrieval-augmented language models. *Advances in Neural Information Processing Systems*, 36: 21573–21612.
- Zheng, K.; Han, J. M.; and Polu, S. 2021. Minif2f: a cross-system benchmark for formal olympiad-level mathematics. *arXiv preprint arXiv:2109.00110*.
- Zhu, K.; Chen, J.; Wang, J.; Gong, N.; Yang, D.; and Xie, X. 2024. Dyval: Dynamic evaluation of large language models for reasoning tasks. In *International Conference on Learning Representations*, volume 2024, 18091–18128.